

Pourquoi le choix d'une base documentaire s'impose-t-il ici face à l'ajout d'une table SQL ?

Vous devez confronter les deux approches sur ce cas d'usage précis.

1. Ce que donnerait une modélisation relationnelle de ces événements, et ses limites concrètes sur ce cas

2. Ce que permet une modélisation documentaire

3. Votre conclusion et le critère qui l'emporte.

Indice pour vous mettre sur la piste, sans vous donner la réponse : un événement de connexion, un paiement, une erreur applicative et un appel d'API ne portent pas du tout les mêmes informations.

SOURCES :

https://www.data-bird.co/blog/sql-vs-nosql?utm_source=website&utm_medium=unknown_page&utm_campaign=general&utm_content=cta_formulaire&utmw=1

<https://www.mongodb.com/docs/manual/data-modeling/>

<https://www.geeksforgeeks.org/system-design/which-database-to-choose-while-designing-a-system-sql-or-nosql/>

Pigeon est une messagerie qui produit des logs mais ne les exploite pas. On souhaite les stocker pour les exploiter, et on parle de plus de 100 000 événements sur 1 an.

Le sujet précise “face à l’ajout d’une table SQL”, ce qui implique que Pigeon utilise déjà du relationnel, sûrement pour ses utilisateurs, abonnements... Pourquoi l’utilisation d’une nouvelle collection noSQL pour ce service de log est plus adaptée :

1. Modélisation relationnelle.

Ce que le relationnel fait très bien, c’est l’intégrité des références grâce aux **clés étrangères**, et les mises à jour cohérentes avec des résultats en cascade.

Or par définition, un événement loggé ne sera **pas modifié ni supprimé** après avoir été écrit.

Admettons qu’on fasse du relationnel, comment architecturer les données :

Une seule table event, avec des colonnes de tous les types? Dans ce cas, pour un événement “connexion”, que contiendrait la colonne “currency”, correspondant à la devise d’un événement paiement? NULL (qui veut dire vide OU inconnu, c’est ambigu)

Imaginons 5 événements avec 6 champs spécifiques, on aurait donc 80% de NULL dans chaque entry, et il faudrait créer 30 contraintes conditionnelles CHECK pour

rendre obligatoire les champs spécifiques aux types d'événements.

Comment **évoluerait** le schéma à l'ajout d'un nouveau type d'événement? Une migration ALTER TABLE ADD COLUMN pour chaque champ : faisable mais peu évolutif, script de migration à versionner, déployer etc. sur la base déjà existante de Pigeon : **coût technique** trop élevé.

Une table par type? On pense alors à la demande métier n°1 "Top 10 des utilisateurs les plus actifs" : si c'est tous événements confondus, quelle serait la requête?

Il faudrait **empiler toutes les tables** avec UNION ALL pour agréger les SELECT : on aurait beaucoup trop de couplage.

Et la demande métier n°4 qui traverse plusieurs types d'événements? Imaginons un gros utilisateur qui a envoyé 2000 messages et payé 12 fois : 24 000 lignes intermédiaires? C'est très peu optimisé.

En MongoDB, **\$match**, **\$group**, **\$sort** sont là pour ça.

2. Modélisation documentaire.

Ce que le non relationnel documentaire fait très bien, c'est la **simplicité** et la **flexibilité** : pas de schéma rigide, les documents peuvent avoir des formats différents, ici adapté à des événements différents. Dans une seule collection events, on va pouvoir faire exister des documents avec des champs communs (id, timestamp...) et des champs spécifiques à leur type.

SQL vérifie la structure au moment de l'écriture et rejette en cas de non respect, alors que noSQL accepte des formats différents, la validation va se faire côté Java Spring Boot avec notre classe de création d'événements.

Le document va aussi être **autonome** : il se lit sans jointure, toutes les informations sont au même endroit.

Pas de clés étrangères obligatoires qui connectent les tables entre elles : la plupart de ces événements n'auront aucune raison métier d'être lié à un autre, l'aspect "relationnel" du SQL n'apporte donc ici aucune réponse à un besoin.

Beaucoup des événements vont être liés à un userId : si on le supprime en cascade avec du SQL, tous ses logs disparaissent? Un log est un **historique**, il ne doit pas disparaître même après la suppression d'un utilisateur.

3. Conclusion

LE critère le plus important qui fait qu'on choisit du noSQL, qui vient de l'indice du sujet : les événements n'auront **pas les mêmes champs** selon leurs types, et les **analyses** métier **transverses** qui portent sur plusieurs types à la fois ont besoin de flexibilité pour les parcourir.